

Temps 小白指南

一个 Rust 二进制 + 一个 Postgres = 你的"私人 Vercel + Sentry + PostHog + Pingdom + Resend + E2B"

Rust + Axum + Sea-ORM

Pingora 代理引擎

PostgreSQL + TimescaleDB

MIT OR Apache-2.0 双许可

AI 原生 • 440+ CLI 命令 • 13 个官方 Claude 技能包

上游仓库 github.com/gotempsh/temps (实测 commit 6f74680, 2026-10)

拾遗任务 acc007e7 · 来源: 抖音"爱科技AI"《一个 Rust 二进制 + 一个 Postgres...》· 2026-10-03 产出

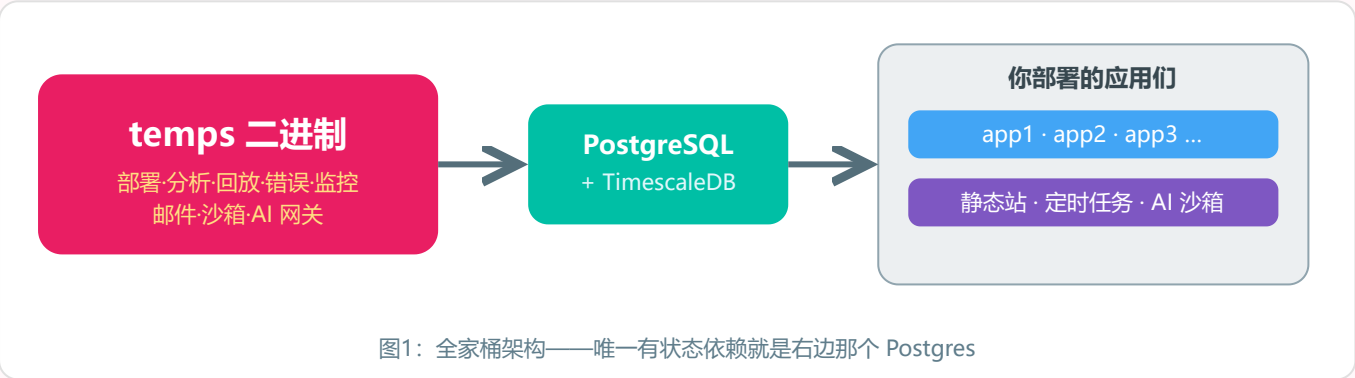
目录 = 12 个疑问 (工科毕业生第一次见到 Temps)

1. Temps 是什么? 一句话说清
2. 为什么敢说"一个二进制 + 一个 Postgres"?
3. 94 个 crate 和"单二进制"矛盾吗?
4. git push 之后到底发生了什么?
5. Pingora 是什么? 为什么不用 Nginx?
6. 它怎么"托管"我的数据库?
7. 网站分析+会话回放怎么塞进一个进程?
8. "Sentry 兼容"是什么意思?
9. "AI 原生"到底原生在哪?
10. AI 网关 BYOK 是怎么省钱的?
11. 一台服务器扛不住怎么办? 多节点!
12. 对比 Coolify / Dokploy / Vercel, 我该选谁?

1 Temps 是什么？一句话说清

Temps 是一个用 Rust 写的自托管 PaaS：你给它一个 Git 仓库，它帮你构建、部署、上 HTTPS、做监控——顺带把你平时按月付费的一堆 SaaS 全都“吃”进自己肚子里：

Temps 内置能力	替你省掉的服务	省多少/月
Git 部署 + 预览 URL	Vercel / Netlify / Railway	\$20+
网站分析 + 漏斗 + 会话回放	PostHog / Plausible / FullStory	\$0–2000
错误追踪 (Sentry 兼容)	Sentry	\$26+
链路追踪/指标/日志 (OpenTelemetry)	Datadog / Grafana Cloud	\$0–500
可用性监控	Pingdom / UptimeRobot	\$20+
托管 Postgres / Redis / S3 / MongoDB	AWS RDS / ElastiCache	\$50+
事务邮件 + DKIM	Resend / SendGrid	\$20+
AI 代码沙箱 + AI 网关	E2B / OpenRouter / LangSmith	\$150+



PaaS (Platform as a Service, 平台即服务)：你只管交代码，构建/运行/扩容/流量入口平台全包。Vercel 就是典型的托管 PaaS，Temps 是它的开源自托管版。

2 为什么敢说"一个二进制 + 一个 Postgres"?

大多数自托管平台一装就是十几个容器：前端、后端、数据库、缓存、队列、消息中间件、网关.....Temps 反着来——除了 Postgres，其他一切（API、代理、后台任务、分析管道）都住在同一个 Rust 进程里。

传统拼装：数据库 + Redis + RabbitMQ + Nginx + 前端容器 + 后端容器.....每个组件都是要升级、备份、监控的"住户"，一台机器像个合租房。

Temps：队列？Postgres 表。缓存？Postgres。任务调度？Postgres 的行级锁轮询。状态只有一个家，备份 = 备份一个库。

别人家的自托管 (7+ 容器)



Temps (2 件套)



图2：从"合租房"到"两件套"——组件少了，运维面就小了

这个思路有专门的名字：**Postgres as the Platform**——把队列（如 pgmq）、发布订阅（LISTEN/NOTIFY）、缓存、定时任务全部交给 Postgres 承担。代价是 Postgres 成为单点命门，收益是备份/迁移/安全边界只剩一条。官方安装器连队列中间件都不装，就是这条路线的极致体现。

3 94 个 crate 和"单二进制"矛盾吗?

不矛盾, 这正是 Rust 的看家本领。 **Cargo workspace**: 一个仓库里放几十个子包 (crate), 开发时各管各的、编译可并行; 发布时 **静态链接成一个可执行文件**——模块化在"源码层", 不拆"进程层"。

实验1实测 (exp1_arch_anatomy.py, 真跑):

workspace 成员总数: 94 个 crate, 按领域分 9 类:

core(platform 内核)	19 个	← 数据库/迁移/实体/队列/配置/插件SDK
observability(可观测性)	11 个	← ClickHouse/日志/指标/OTel/状态页
ops(错误追踪/邮件/安全)	10 个	
network(代理/DNS/TLS)	10 个	
ai(网关/沙箱/MCP)	9 个	
deploy(Git 构建部署)	9 个	
import(平台迁移导入)	9 个	← 能从 Coolify/Dokploy/CapRover/K8s 整体搬家
resources(托管库/存储)	8 个	← temps-query-postgres/redis/s3/mongodb
analytics(分析/回放)	6 个	

含 main.rs 的目标 4 个 (cli/preview-gateway/pty-agent/vm-agent),

但发布产物只有 1 个 temps 二进制——temps-cli 是"单一入口"。

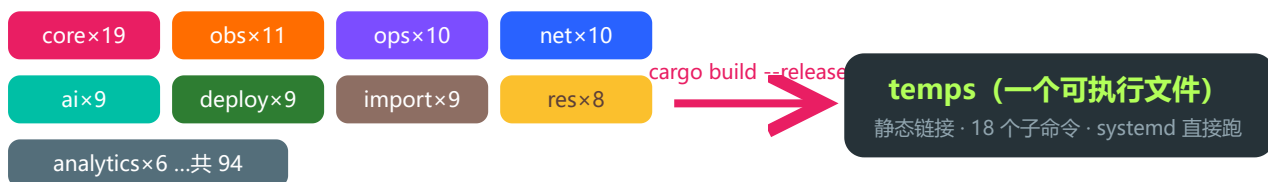


图3: 源码里 94 个模块, 硬盘上只有一个文件

实测这个单一二进制的 18 个子命令: **serve** (API 服务器, 内嵌控制台 UI)、**proxy** (Pingora 流量入口)、**migrate**、**backup**、**doctor** (自检)、**join/agent/node** (多节点集群)、**network**、**edge** (边缘 CDN 节点)、**sandbox/firecracker** (microVM 沙箱) **一个二进制既是服务器又是运维工具箱**。

4 git push 之后到底发生了什么？

这是 PaaS 的灵魂流程。Temps 版本：



全程只需 Postgres 记账：部署任务 = 数据库行，worker 进程轮询认领（同一二进制里的后台线程）

没有消息队列中间件、没有独立的 CI 服务器——“排队”就是一张表 + SELECT ... FOR UPDATE SKIP LOCKED

构建产物是 Docker 镜像；运行时 Pingora 把你的域名按路由表转发到对应容器端口

图4：push-to-deploy 五步流水线，队列表就是 Postgres

预览部署（Preview Deployment）：每个分支/PR 都给一个独立的临时网址，合并即销毁——审查界面改动不用本地起环境。**零停机发布**：先起新版本容器，健康检查通过后才把流量切过去，旧容器再退场。

5 Pingora 是什么？为什么不用 Nginx？

Pingora 是 Cloudflare 开源的 Rust 网络代理框架（支撑 Cloudflare 每天上万亿次请求的那套引擎）。Temps 把它作为“流量大门”：**所有进来的 HTTP(S) 请求先过 Pingora，再按域名路由表转发到对应应用容器。**

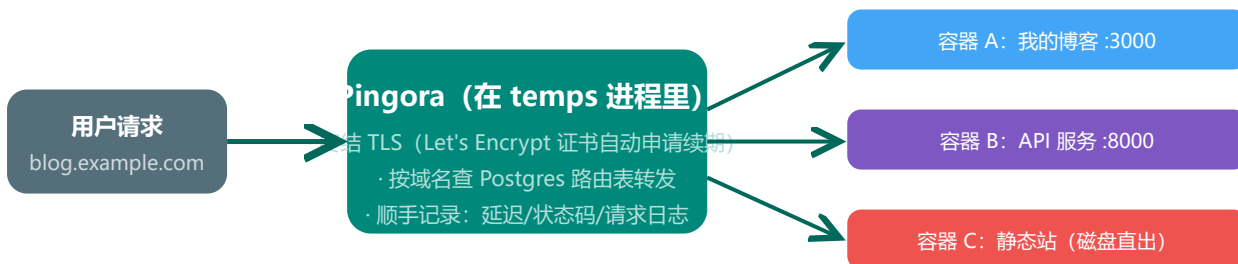


图5：一个进程自带“负载均衡 + 反向代理 + CDN 边缘节点”（edge 子命令）

为什么不用 Nginx？因为 Nginx 是“配置文件驱动”的外部进程，Temps 需要**路由表跟着数据库实时变**（用户一点按钮就加域名/加路由）。Pingora 是 Rust 库，直接嵌在自家进程里编程，还能多路复用连接、复用 Cloudflare 生产验证的性能。类比已学的 [[负载均衡与反向代理]]：同一概念，从“独立软件”变成“进程内的库”。

6 它怎么"托管"我的数据库?

Temps 的 `temps-query-*` 系列 crate (postgres/redis/s3/mongodb) 做的事: 你在控制台点一下"开通 Postgres", 它就用 **Bollard** (Docker 的 Rust API 客户端) 拉起对应容器, 设置好密码、网络、备份计划, 再把连接串注入你应用的 **环境变量**。销毁时连数据卷一起回收。

一句话: 把 AWS RDS 的"控制台点两下就有数据库"体验搬到你自己机器上——只不过"控制面"和"数据面"都在同一个 temps 二进制里, 数据库本体是普通 Docker 容器。

注意区分两层: Temps **托管**的数据库 (给你的应用用, Redis/MinIO/Mongo 都能一键开) 和 Temps **自己用的**那一个 Postgres (TimescaleDB, 存平台一切状态)。**前者随便开几十个, 后者全场只有这一个, 且不可替代。**

7 网站分析 + 会话回放怎么塞进一个进程?

网页里嵌一小段 `<script>` (官方 SDK `@temps-sdk/react-analytics` 包一层 Provider 即可), 每个访客行为打点发送到 Pingora 捕获的分析端点:

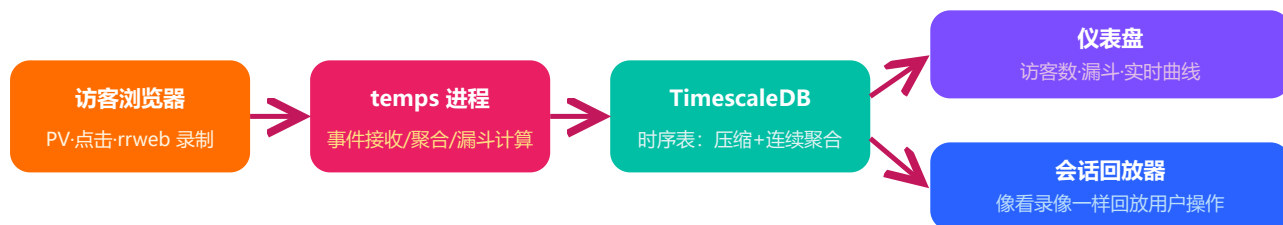


图6: PostHog 级别的分析管线, 一个进程 + 一个库搞定

会话回放 (Session Replay): 前端用 **rrweb** 把 DOM 变化序列录下来 (不是视频, 是"操作日志"), 回放时逐帧重建页面——所以用户"点了哪里、看到什么"都能精确重现, 排查转化流失利器。**TimescaleDB** 是 Postgres 的时序扩展: 海量打点事件自动压缩、按小时/天预聚合 (连续聚合), 分析查询不用扫原始明细。

8 "Sentry 兼容"是什么意思?

实验2 实测 Temps 的 OpenAPI 里有 `/0/organizations/{org}/releases/` 这类路径——**这就是 Sentry 的 API 形状。**

Sentry SDK 只需要改一个 DSN (数据收录地址) 指向你的 Temps, 其他一行代码不动, 错误上报照常工作。

这套"协议兼容吃生态"的打法在 Temps 里出现了四次: Sentry 兼容 (错误追踪)、`@temps-sdk/sandbox` 兼容

`@vercel/sandbox` (沙箱 SDK)、OpenAI 兼容端点 (AI 网关)、OpenTelemetry (链路追踪本来就是标准)。**协议面标准化 = 生态位免费继承**——与知识库里 [[Realtime协议兼容层]]、[[MCP协议]] 是同一个思想的不同战场。

9 "AI 原生"到底原生在哪？

这是 Temps 最有趣、也最对当下胃口的设计。四件事：

- ① **每个控制台操作都有 CLI 命令**。实测（实验2）npm CLI 有 **77 个命令组、143 个命令实现文件**（官方口径"69 组 / 440+ 条命令"是按子命令展开数统计）。部署、看追踪、迁移数据库、加域名——全部 `bunx @temps-sdk/cli ...` 可做。命令行不只是人的工具，更是 **给 AI 程序化调用准备的操作面**。
- ② **13 个官方 drop-in 技能包**。仓库 `skills/` 目录实测 13 个（deploy-to-temps、temps-cli、temps-mcp-setup、estimate-temps-savings...），直接拷进 `.claude/skills/`，Claude Code/Codex/OpenCode 就"学会"操作 Temps——和已学的 [[AgentSkills技能包]] 完全同构：**把使用手册写成 AI 能执行的说明书**。
- ③ **内置 MCP server + AI 对话**。对接你的可观测数据（追踪/指标/告警/收入），默认只读、写操作先出方案等你确认——安全边界设计值得抄。
- ④ **工作流沙箱**。Temps 能替你跑 Claude Code：给一个仓库和任务，它在 microVM 沙箱里执行 agent 并自动注入平台技能。



图7：AI 原生三件套——命令即 API、技能即手册、协议即入口

10 AI 网关 BYOK 是怎么省钱的？

BYOK (Bring Your Own Key, 自带密钥)：你各家模型服务商 (OpenAI/Anthropic/xAI/Gemini) 的 API Key 存在你自己的服务器上（加密落库），应用统一调一个 **OpenAI 兼容端点**——换模型只改 base URL，代码零改动。

省钱之外更值钱的是 **用量归因**：每个请求记录 token 数、延迟、错误率、按模型折算成本，谁是花钱大户一目了然。这本质上是知识库 [[Agent网关与消息通道]] 思想的平台化：M 个应用 × N 个服务商的 M×N 集成，被一个网关压成 M+N。

11 一台服务器扛不住怎么办？多节点！

单二进制不等于单机。实测子命令里的集群三兄弟：`temps join`（把新机器加入集群当 worker）、`temps agent`（worker 上的代理进程）、`temps node`（管理节点：list/show/drain/remove）。

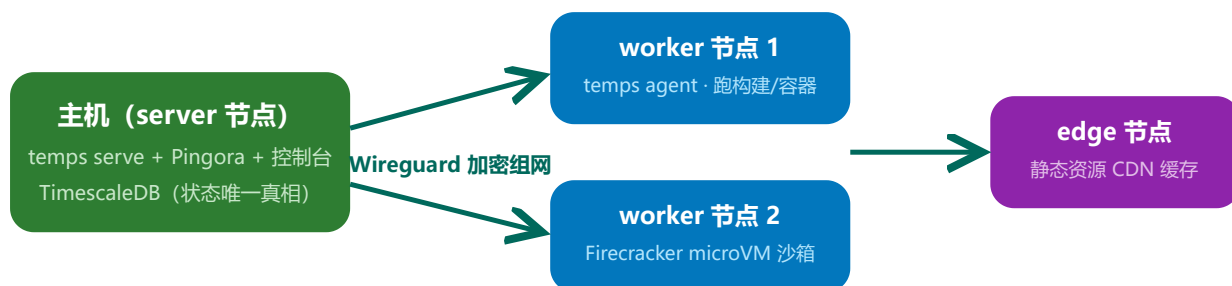


图8: server/worker/edge 三种角色，一个二进制谁都能演 (ADR-048: 沙箱上 worker 节点)

worker 之间用 **WireGuard**（内核级 VPN）组加密 overlay；沙箱后端可选 **Firecracker**（AWS Lambda 同款 microVM，毫秒级启动的极轻虚拟机）。**架构模式 = 控制面 (server) + 数据面 (worker/edge) 分离**，与 [[控制面与数据面]]、[[机器人守护进程架构]] 的分层思想同源。

12 对比 Coolify / Dokploy / Vercel，我该选谁？

维度	Temps	Coolify	Dokploy	Vercel (托管)
形态	单二进制 + Postgres	Docker 多容器	Docker 多容器	黑盒平台
开源许可	MIT OR Apache-2.0	Apache-2.0	Apache-2.0	专有
Git push 部署/预览	✓	✓	✓	✓
分析/回放/错误追踪	✓ 内置	✗	✗	付费插件
一键模板库	✗ (暂无)	✓ 280+	✓	✓
边缘网络/DDoS 吸收	✗	✗	✗	✓
AI 原生 (CLI/skills/沙箱)	✓ 最强	✗	✗	部分
生态成熟度	最年轻 (440+★)	56k+★	成熟	最大

选型口诀：要“全家桶一体机 + AI agent 运维”选 Temps；要模板多、社区大选 Coolify；要纯托管省心选 Vercel——**Temps 的差异化不在“能部署”，而在“部署+观测+AI 运维长在一个进程里”**。另外它内置 Coolify/Dokploy/K8s/Portainer 导入器，搬家成本几乎为零。

★ 这个项目给我什么（能力清单）

① **自托管全栈平台模板：**一台 VPS 一条命令拥有 Vercel 级体验；② **零成本可观测栈：**分析/回放/错误/追踪不再按量付费；③ **AI 运维样本：**13 个官方技能包 + 440+ 命令是“给 agent 造产品”的活教材；④ **Postgres as Platform 范例：**学它怎么把队列/缓存/调度全塞进 PG；⑤ **Rust workspace 工程范本：**94 crate 单二进制的模块化组织方式；⑥ **协议兼容策略库：**Sentry/Vercel/OpenAI 兼容层怎么“偷”生态。

🔧 主实验：三连拆——从源码验证"一个 Rust 二进制 + 一个 Postgres" (已真跑，输出见 exercise\exp*_out.txt)

步骤 1 克隆仓库并写解剖器：`git clone --depth 1 https://github.com/gotempsh/temps.git temps/repo`，编写 `exercise\exp1_arch_anatomy.py` (纯标准库)。

目的：数清 workspace 结构与二进制目标。**怎么验证：**python 运行后与 README 说法交叉核对。

✅ 预期/实测结果：94 个 crate 分 9 大领域；4 个 main.rs 但发布单一 temps 二进制；18 个子命令枚举与源码一致。

步骤 2 验证 "AI 原生" 量化口径：编写 `exp2_cli_skills_count.py`，数 commands/ 子目录、openapi.json 路径、skills/ 目录。

✅ 预期/实测结果：77 命令组 / 143 命令文件 (README 口径 69 组 440+ 条为子命令展开数，已在 PDF 标注差异)；OpenAPI 853 路径含 Sentry 兼容 /0/ 风格；13 个官方 drop-in skills。

步骤 3 解剖官方安装器：`curl -fsSL https://temps.sh/deploy.sh -o deploy_downloaded.sh`，再跑 `exp3_deploy_anatomy.py` 提取步骤与安装清单。

✅ 预期/实测结果：4418 行 TUI 向导 9 个步骤函数；只装两样——GitHub Releases 的 temps 二进制 (systemd 托管) + timescale/timescaledb-ha:pg18 容器；核心函数 `install_temps_binary()` 下载 `temps-$target.tar.gz`。至此"一个 Rust 二进制 + 一个 Postgres"实锤。

⚠️ 坑与说明：本地未跑 `temps serve` 全功能 (需 Docker + Linux systemd, Windows 仅分析层面验证)；CLI 官方宣称 440+ 命令与源码文件数 (143) 口径不同——前者数 subcommand 展开，后者数实现文件，均已如实记录。

知识库沉淀 (vault\)：新概念 [[单二进制自托管架构]]、[[Postgres即平台]]、[[BYOK模型网关与用量归因]]；关联已有 [[AgentSkills技能包]]、[[负载均衡与反向代理]]、[[控制面与数据面]]、[[提交即部署]]、[[内容型开源与双许可]]。项目笔记：`vault\项目笔记\temps.md`。